



Other Operating Systems



Luís Miguel Pinho

Real-time Operating Systems

- ▶ We have been using Linux as a learning tool
 - ▶ It provides support to many real-time systems technologies
 - ▶ Fixed Priority scheduling
 - ▶ EDF (CBS)
 - ▶ Priority inheritance
 - ▶ But it is not specifically targeting real-time systems
- ▶ Real-time OSes on the other hand are specifically designed for real-time, with predictability in mind
 - ▶ But in some cases may even have less support to real-time mechanisms
- ▶ Examples
 - ▶ FreeRTOS, Zephyr, ThreadX, NuttX, RTEMS
 - ▶ QNX/Neutrino, VxWorks, PikeOS, LynxOS, Integrity, Deos

FreeRTOS

- ▶ **FreeRTOS is a lightweight microkernel**
 - ▶ No processes, single main application
 - ▶ Application code and kernel executing in the same space
 - ▶ No device drivers isolation
 - ▶ Application is compiled to the execution binary, configuration flags are specified at compile time (FreeRTOSConfig.h)
 - ▶ Simple and easy to use
 - ▶ Supports a relevant number of architectures
- ▶ **Includes small set of libraries**
 - ▶ TCP, MQTT, HTTP, JSON, Cell, AWS IoT, ...
 - ▶ LoRaWAN, FAT, IPv6

FreeRTOS

- ▶ By default, FreeRTOS uses a fixed-priority preemptive scheduling policy
 - ▶ Preemption can be disabled
 - ▶ Round-robin in equal priority tasks with time slices
 - ▶ Although some implementations exist with EDF support, these are research-only
 - ▶ No notion of deadline in the OS
 - ▶ It is a tick-based OS, which may impact for higher responsiveness
 - ▶ Timers use the OS tick – higher resolutions increases overhead
 - ▶ It also provides a co-routine approach for very small processors with RAM constraints, which is no longer developed
 - ▶ There are API variants “fromISR” that can be used from Interrupt Service Routines for more efficient use

FreeRTOS

► Task creation

```
int main(void){
    xTaskCreate(vPeriodicTask, "My_Periodic", configMINIMAL_STACK_SIZE,
               NULL, tskIDLE_PRIORITY + 1, NULL);

    vTaskStartScheduler();
    while(1);
}

void vPeriodicTask( void * pvParameters ) {
    TickType_t xLastWakeTime;
    const TickType_t xDelay3ms = pdMS_TO_TICKS( 3 );
    xLastWakeTime = xTaskGetTickCount();

    for( ;; ) {
        // Do something

        vTaskDelayUntil( &xLastWakeTime, xDelay3ms );
    }
}
```

FreeRTOS

► Task creation (round robin)

```
int main(void){
    xTaskCreate(vPeriodicTask1, "Periodic 1", configMINIMAL_STACK_SIZE,
                NULL, tskIDLE_PRIORITY + 2, NULL);
    xTaskCreate(vPeriodicTask2, "Periodic 2", configMINIMAL_STACK_SIZE,
                NULL, tskIDLE_PRIORITY + 2, NULL);
    xTaskCreate(vSporadic, "Sporadic", configMINIMAL_STACK_SIZE,
                NULL, tskIDLE_PRIORITY + 2, NULL);

    vTaskStartScheduler();
    while(1);
}
```

FreeRTOS

▶ Task communication

- ▶ Queues are the primary form of inter task/interrupt communication, can be used to send data or to synchronize
- ▶ In most cases they are used as thread safe FIFO (First In First Out) buffers with new data being sent to the back of the queue, although data can also be sent to the front
- ▶ Reading/Writing from/to an empty/full queue will block until either data/space becomes available on the queue, or the block time expires
- ▶ If more than one task block on the same queue then the task with the highest priority will be the task that is unblocked first.

FreeRTOS

► Queue example

```
QueueHandle_t xQueue;

void vSenderTask(void *pvParameters);
void vReceiverTask(void *pvParameters);

int main(void)
{
    xQueue = xQueueCreate(5, sizeof(int));

    if (xQueue != NULL)
    {
        xTaskCreate(vSenderTask, "Sender", configMINIMAL_STACK_SIZE, NULL,
                    tskIDLE_PRIORITY + 1, NULL);
        xTaskCreate(vReceiverTask, "Receiver", configMINIMAL_STACK_SIZE, NULL,
                    tskIDLE_PRIORITY + 1, NULL);

        // Start the scheduler
        vTaskStartScheduler();
    }

    for (;;)
    }
```

```
void vSenderTask(void *pvParameters)
{
    int iValueToSend = 0;

    for (;;)
    {
        xQueueSend(xQueue, &iValueToSend, portMAX_DELAY);

        iValueToSend++;

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// Receiver task function implementation
void vReceiverTask(void *pvParameters)
{
    int iReceivedValue;

    for (;;)
    {
        xQueueReceive(xQueue, &iReceivedValue, portMAX_DELAY);

        printf("Received: %d\n", iReceivedValue);
    }
}
```

FreeRTOS

- ▶ Synchronization can be achieved with Semaphores and Mutexes
 - ▶ Binary semaphores allow for mutual exclusion and synchronization, it can be either free or taken
 - ▶ Task that tries to Take a taken semaphore blocks until the semaphore is released or timeout
 - ▶ Mutex is better for mutual exclusion as it provides priority inheritance
 - ▶ Counting semaphores allow for more complex synchronisation mechanisms to be implemented, with multiple tasks able to take the same semaphore

FreeRTOS

► Binary semaphore example

```
// Define the semaphore handle
SemaphoreHandle_t xBinarySemaphore;

// Task function prototype
void vTaskFunction(void *pvParameters);

// Interrupt service routine prototype
void vISRHandler(void);

int main(void)
{
    // Create the binary semaphore
    xBinarySemaphore = xSemaphoreCreateBinary();

    // ...
```

```
void vTaskFunction(void *pvParameters)
{
    for (;;)
    {
        // Wait for the semaphore to be given by the ISR
        if (xSemaphoreTake(xBinarySemaphore, portMAX_DELAY) == pdTRUE)
        {
            // Task code goes here
        }
    }
}

// Interrupt service routine implementation
void vISRHandler(void)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // Give the semaphore to unblock the task
    xSemaphoreGiveFromISR(xBinarySemaphore, &xHigherPriorityTaskWoken);

    // Perform a context switch if necessary
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}
```

FreeRTOS

► Mutex example

```
SemaphoreHandle_t xMutex;

int main(void)
{
    // Create the mutex
    xMutex = xSemaphoreCreateMutex();

    if (xMutex != NULL)
    {
        ...
    }
}

void vTask1(void *pvParameters)
{
    for (;;)
    {
        // Attempt to take the mutex
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
        {
            // Critical section: access shared resource
            // ...

            // Release the mutex
            xSemaphoreGive(xMutex);
        }

        // Delay for a period
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void vTask2(void *pvParameters)
{
    for (;;)
    {
        // Attempt to take the mutex
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
        {
            // Critical section: access shared resource
            // ...

            // Release the mutex
            xSemaphoreGive(xMutex);
        }

        // Delay for a period
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

FreeRTOS

- ▶ Synchronization can be achieved with Semaphores and Mutexes
 - ▶ Binary semaphores allow for mutual exclusion and synchronization, it can be either free or taken
 - ▶ Task that tries to Take a taken semaphore blocks until the semaphore is released or timeout
 - ▶ Mutex is better for mutual exclusion as it provides priority inheritance
 - ▶ Counting semaphores allow for more complex synchronisation mechanisms to be implemented, with multiple tasks able to take the same semaphore
- ▶ It is also possible to use direct task notifications
 - ▶ A direct to task notification is an event sent directly to a task, rather than indirectly to a task via an intermediary object such as a queue, event group or semaphore
 - ▶ Each RTOS task has an array of task notifications, the task can block waiting for the notification

FreeRTOS

► Task notification example

```
// Task handles
TaskHandle_t xTask1Handle = NULL;
TaskHandle_t xTask2Handle = NULL;

// Main function
int main(void)
{
    // Create the tasks
    xTaskCreate(vTask1, "Task1", configMINIMAL_STACK_SIZE, NULL,
               tskIDLE_PRIORITY + 1, &xTask1Handle);
    xTaskCreate(vTask2, "Task2", configMINIMAL_STACK_SIZE, NULL,
               tskIDLE_PRIORITY + 1, &xTask2Handle);

    ...
}
```

```
// Task 1 function implementation
void vTask1(void *pvParameters)
{
    for (;;)
    {
        // Perform some work
        // ...

        // Notify Task 2
        xTaskNotifyGive(xTask2Handle);

        // Delay for a period
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// Task 2 function implementation
void vTask2(void *pvParameters)
{
    for (;;)
    {
        // Wait for notification from Task 1
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

        // Perform some work after being notified
        // ...
    }
}
```

Zephyr

▶ Scalable RTOS

- ▶ Small, but slightly larger than FreeRTOS due to its additional features
- ▶ Growing community and growing architecture support
- ▶ A more feature-rich and scalable platform with support for a wider range of hardware architectures
 - ▶ Allows for customization by including or excluding features as needed
 - ▶ Includes multi-threading, device drivers, and networking stacks
 - TCP/IP, Bluetooth Low Energy, USB, LoRa
 - ▶ And many other features
 - File system, shell, virtualization support, crypto, ...
- ▶ Provides memory protection
- ▶ Still compile-time based configuration

Zephyr

- ▶ Supports multiple scheduling algorithms
 - ▶ Fixed-priority is the base algorithm
 - ▶ Threads of equal priority are not per default time sliced
 - ▶ Scheduler selects the one waiting longer, which will execute until explicitly yielding the processor
 - ▶ However, time slicing can be enabled
 - ▶ Supports EDF within threads of the same priority
 - ▶ Limited form of hierarchical scheduling
 - ▶ Supports cooperative threads, which execute until explicitly yield the processor
 - ▶ Cooperative scheduling / limited preemptive scheduling
 - ▶ Supports workqueue threads
 - ▶ Dedicated thread to process work items queued in a first in, first out manner

Zephyr

► Thread creation

```
#define STACK_SIZE 512
#define PRIORITY 5

K_THREAD_STACK_DEFINE(my_stack_area, STACK_SIZE);
struct k_thread my_thread;

void my_thread_func(void *arg1, void *arg2, void *arg3) {
    uint64_t next_wakeup = k_uptime_get() + K_SECONDS(1);

    while (1) {
        // Perform the periodic task here
        ...

        // Sleep until the next wakeup time
        k_sleep(K_TIMEOUT_ABS_MS(next_wakeup));

        // Calculate the next wakeup time
        next_wakeup += K_SECONDS(1);
    }
}

main(){
    ...
    k_tid_t my_tid = k_thread_create(&my_thread, my_stack_area, STACK_SIZE, my_thread_func,
                                     NULL, NULL, NULL, PRIORITY, 0, K_NO_WAIT);
```

Zephyr

▶ Thread communication

▶ Supports multiple types of inter thread communication objects

▶ FIFO and LIFO queues

- Block if queue empty (or timeout)

▶ Circular buffer (Message queue/pipe)

- Block if full/empty (or timeout)

▶ Stack array

▶ Bidirectional queue (Mailbox)

- Messages can be exchanged synchronously (sending thread blocks waiting for reception) or asynchronously
- Receiving threads may block waiting for a message
- A Mailbox can be shared by multiple sending/receiving threads

Zephyr

► Example of message queue

```
struct k_msgq my_msgq;  
#define MSGQ_SIZE 10  
#define MSG_SIZE sizeof(int)  
char __aligned(4) buffer[MSGQ_SIZE * MSG_SIZE];
```

...

```
k_msgq_init(&my_msgq, buffer,  
            MSG_SIZE, MSGQ_SIZE);
```

```
void sender_thread_func(void *arg1, void *arg2, void *arg3) {  
    int msg = 42;  
    while (1) {  
        k_msgq_put(&my_msgq, &msg, K_NO_WAIT);  
        printk("Message sent: %d\n", msg);  
        k_sleep(K_SECONDS(1));  
    }  
}
```

```
void receiver_thread_func(void *arg1, void *arg2, void *arg3) {  
    int received_msg;  
    while (1) {  
        k_msgq_get(&my_msgq, &received_msg, K_FOREVER);  
        printk("Message received: %d\n", received_msg);  
    }  
}
```

Zephyr

► Example of message box

```
struct k_mbox my_mbox;
```

```
k_mbox_init(&my_mbox);
```

```
void sender_thread_func(void *arg1, void *arg2, void *arg3) {  
    struct k_mbox_msg msg;  
    int data = 42;  
    msg.info = 0;  
    msg.tx_data = &data;  
    msg.tx_block.data = NULL;  
    msg.tx_target_thread = K_ANY;  
    while (1) {  
        k_mbox_put(&my_mbox, &msg, K_NO_WAIT);  
        printk("Message sent: %d\n", data);  
        k_sleep(K_SECONDS(1));  
    }  
}
```

```
void receiver_thread_func(void *arg1, void *arg2, void *arg3) {  
    struct k_mbox_msg msg;  
    int received_data;  
  
    msg.rx_source_thread = sender_tid;  
    msg.rx_block.data = NULL;  
  
    while (1) {  
        k_mbox_get(&my_mbox, &msg, &received_data,  
                  K_FOREVER);  
        printk("Message received: %d\n", received_data);  
    }  
}
```

Zephyr

▶ Thread synchronization

▶ Semaphores (counting)

- ▶ A thread may choose to wait for the semaphore to be available. Any number of threads may wait on an unavailable semaphore simultaneously. When the semaphore is given, it is taken by the highest priority thread that has waited longest.

▶ Mutexes

- ▶ Supports priority inheritance (but no support for EDF)

▶ Conditional variables

- ▶ Enables threads to wait until a particular condition occurs, requires a mutex

▶ Events

- ▶ One or more threads may wait on an event object until the desired set of events has been delivered to the event object. When new events are delivered to the event object, all threads whose wait conditions have been satisfied become ready simultaneously.

Zephyr

► Semaphore example for notification

```
struct k_sem my_sem;
```

```
k_sem_init(&my_sem, 0, 10); // Initial count is 0, maximum count is 10
```

```
void sender_thread_func(void *arg1, void *arg2, void *arg3) {  
    while (1) {  
        // Perform some work  
        ...  
  
        // Give the semaphore  
        k_sem_give(&my_sem);  
  
        // Delay for a period  
        k_sleep(K_SECONDS(1));  
    }  
}
```

```
void receiver_thread_func(void *arg1, void *arg2, void *arg3) {  
    while (1) {  
        // Take the semaphore  
        k_sem_take(&my_sem, K_FOREVER);  
  
        // Perform some work after taking the semaphore  
        ....  
    }  
}
```

Zephyr

► Semaphore example for exclusion

```
struct k_sem my_sem;
```

```
k_sem_init(&my_sem, 1, 1); // Initial count is 1, maximum count is 1
```

```
void thread_1_func(void *arg1, void *arg2, void *arg3) {  
    while (1) {  
        k_sem_take(&my_sem, K_FOREVER);  
  
        ...  
        k_sem_give(&my_sem);  
  
        ...  
    }  
}
```

```
void thread_2_func(void *arg1, void *arg2, void *arg3) {  
    while (1) {  
        k_sem_take(&my_sem, K_FOREVER);  
  
        ...  
        k_sem_give(&my_sem);  
  
        ...  
    }  
}
```

Zephyr

► Mutex example

```
struct k_mutex my_mutex;
```

```
k_mutex_init(&my_mutex);
```

```
void thread1_func(void *arg1, void *arg2, void *arg3) {  
    while (1) {  
        // Take the mutex  
        k_mutex_lock(&my_mutex, K_FOREVER);  
  
        // Critical section: access shared resource  
        ...  
  
        // Release the mutex  
        k_mutex_unlock(&my_mutex);  
  
        ...  
    }  
}
```

```
void thread2_func(void *arg1, void *arg2, void *arg3) {  
    while (1) {  
        // Take the mutex  
        k_mutex_lock(&my_mutex, K_FOREVER);  
  
        // Critical section: access shared resource  
        ...  
  
        // Release the mutex  
        k_mutex_unlock(&my_mutex);  
  
        ...  
    }  
}
```

Zephyr

► Conditional variable example

```
struct k_condvar my_condvar;
struct k_mutex my_mutex;

k_condvar_init(&my_condvar);
k_mutex_init(&my_mutex);

void thread1_func(void *arg1, void *arg2, void *arg3) {
    while (1) {
        ...
        // Lock the mutex
        k_mutex_lock(&my_mutex, K_FOREVER);

        counter--;

        // Signal the conditional variable
        if (counter==0)
            k_condvar_signal(&my_condvar);

        // Unlock the mutex
        k_mutex_unlock(&my_mutex);

        ...
    }
}

void thread2_func(void *arg1, void *arg2, void *arg3) {
    while (1) {
        // Lock the mutex
        k_mutex_lock(&my_mutex, K_FOREVER);

        // Wait for the conditional variable to be signaled
        while(counter == 0)
            k_condvar_wait(&my_condvar, &my_mutex, K_FOREVER);

        // Unlock the mutex
        k_mutex_unlock(&my_mutex);

        ...
    }
}
```